

第9章

异步串行通信



第9章 异步串行通信

- 9.1 串行通信基本概念
- 9.2 MSP430异步串行通信



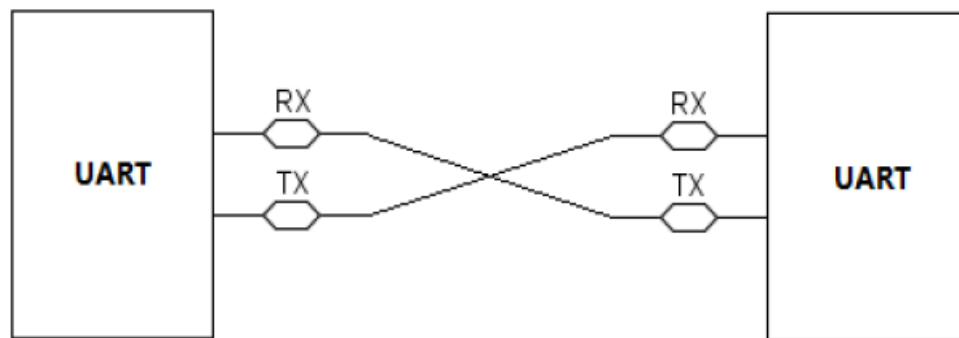
第9章 异步串行通信

- 9.2.1 异步串行通信基本概念
- 9.2.2 MSP430的USCI

9.2.1 异步串行通信基本概念



UART (Universal Asynchronous Receiver/Transmitter) 是通用异步收发器的缩写，一般称为串口。由于不需要时钟线，且为全双工，所以UART有两根数据线，发送Tx和接收Rx。

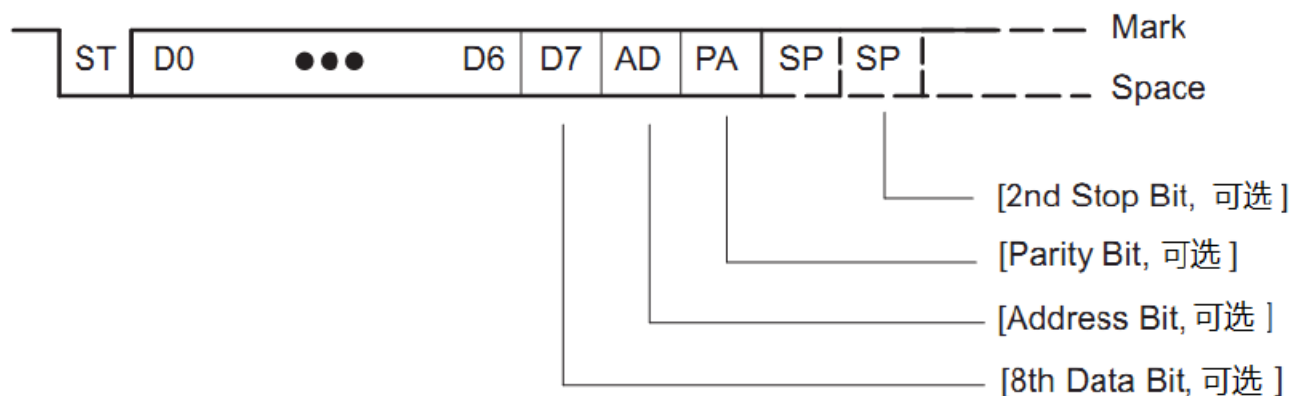


图示是双机进行串口通信的硬件连接，没有公共时钟线且全双工工作就无所谓主机、从机概念，一台设备的接收相当于另一台设备的发送。

9.2.1 异步串行通信基本概念



UART通信协议的**数据帧格式**：首先是一个起始位，然后是7-8位可选的数据位，0-1位可选的地址判别位、0-1位可选的奇偶判别位、1-2位可选的高电平停止位。



- 1) 起始位ST是高电平到低电平的下降沿触发的。也就是每传输7-8位数据都要消耗一个起始位。为什么要起始位呢？这是因为异步通信时，设备双方的时钟一定不相等（差别百万分之一也不行），为了消除累计误差，必须引入起始位。
- 2) 数据位7-8位。为什么会有7或8位的数据长度选择呢？我们知道1个字节是8位，按理说应该都是8位传输才对。其实不是所有类型数据都是8位的，比如ASCII码，7位就够了。假如传输的都是ASCII码，不就赚到了1位吗？能省则省。
- 3) 地址位Address Bit，0表示前面的7或8位是数据，1表示是地址，这在1主多从通信中有用。

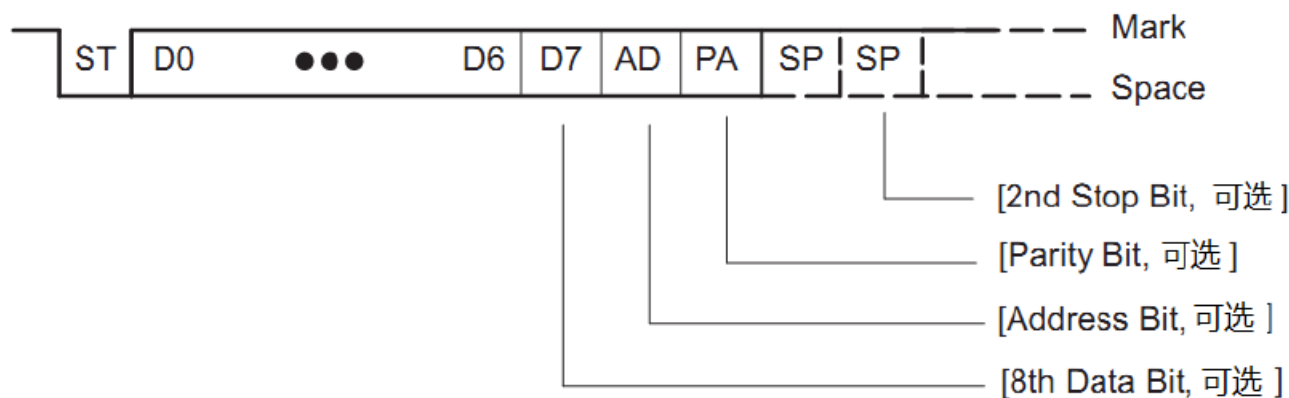
9.2.1 异步串行通信基本概念



4) 奇偶校验位Parity Bit。

5) 停止位SP，数据传输完后，需至少维持高电平1-2位才能重新开始下一帧的传输。当然，歇个10天半月都是高电平再开始下一帧也是可以的，也就是说，1-2位是最少需要的“休息”位。

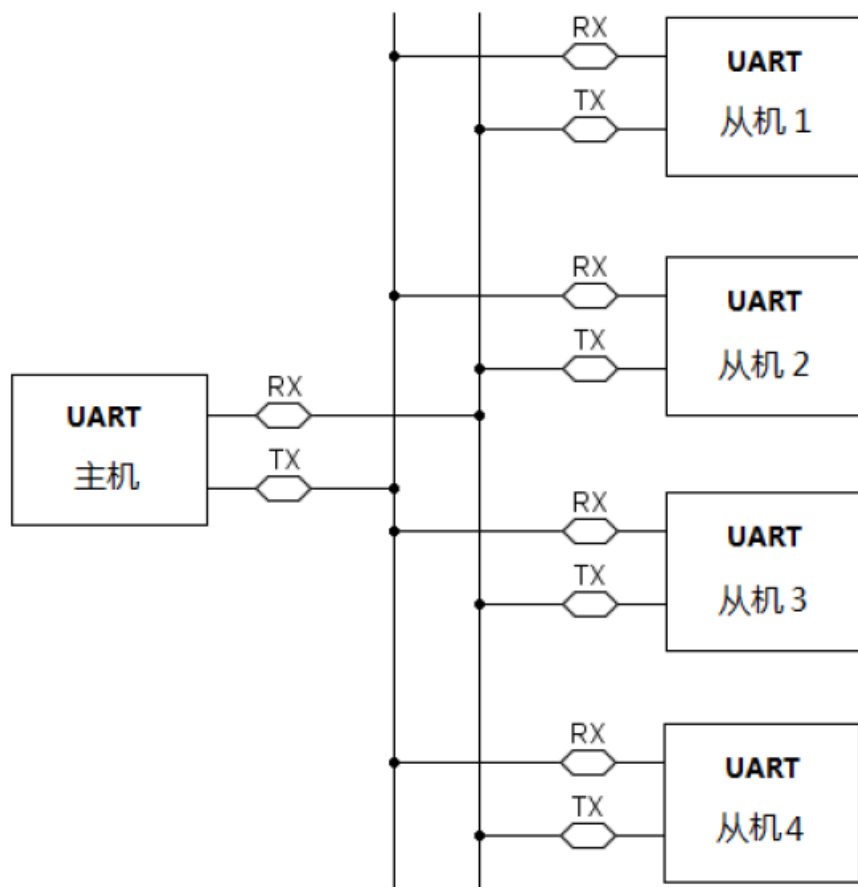
“休息位”也是用来校准异步通信时钟误差的，休息位越长，对两个时钟差值容忍程度越高。根据设备实际数据传输速率和时钟匹配程度的不同，需要最少1位或者2位的停止位。



9.2.1 异步串行通信基本概念



UART可以1主多从进行通信：主机的Tx连上全部从机的Rx，主机的Rx连上所有从机的Tx



一主多从的基本工作原理：

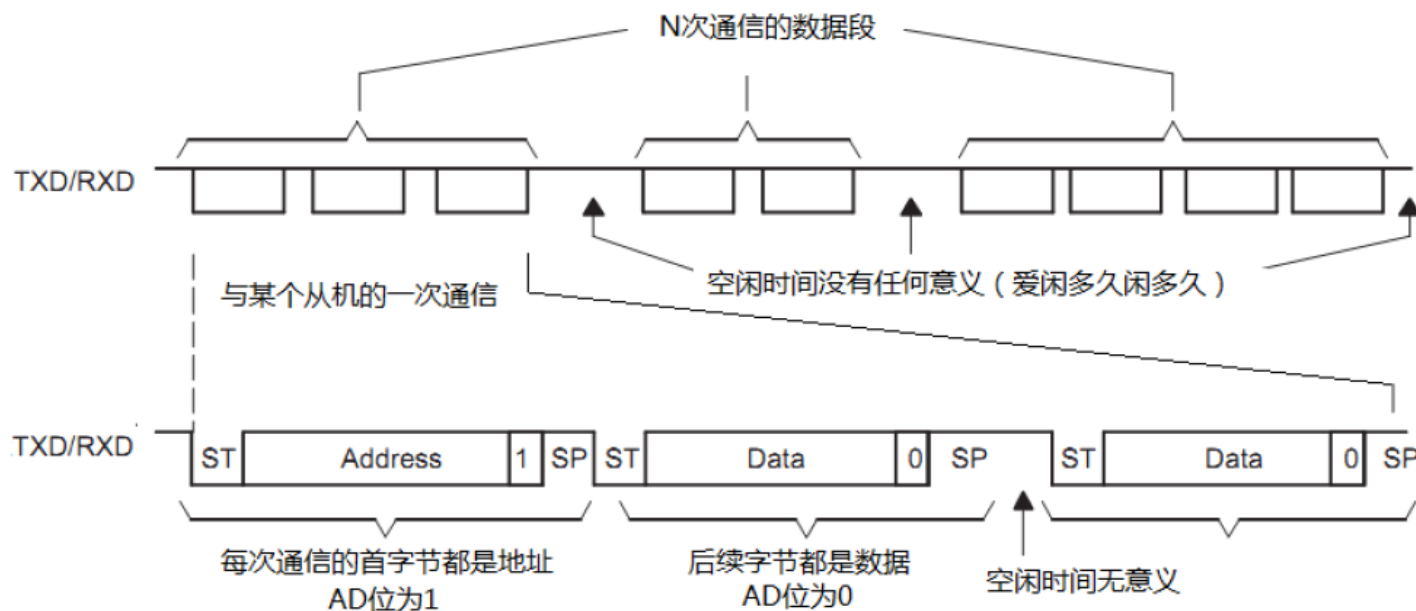
- 1) 所有从机只能与主机通信。每次只能1个从机与主机通信，从机之间想传递信息，只能间接通过主机“捎话”。
- 2) 主机先发送地址帧，所有从机都接收，并判断是不是“喊自己”，要是喊自己，则开始通信。其他从机继续“监听”主从通信是否完毕。
- 3) 如果主从通信完毕，则大家又打起精神看是否有“面圣”的机会。
- 4) 按照判别数据/地址帧的方法来区分，有地址位和空闲帧两种方法。

9.2.1 异步串行通信基本概念



地址位模式多机通信： 在每帧中插入了一个地址位Address Bit。

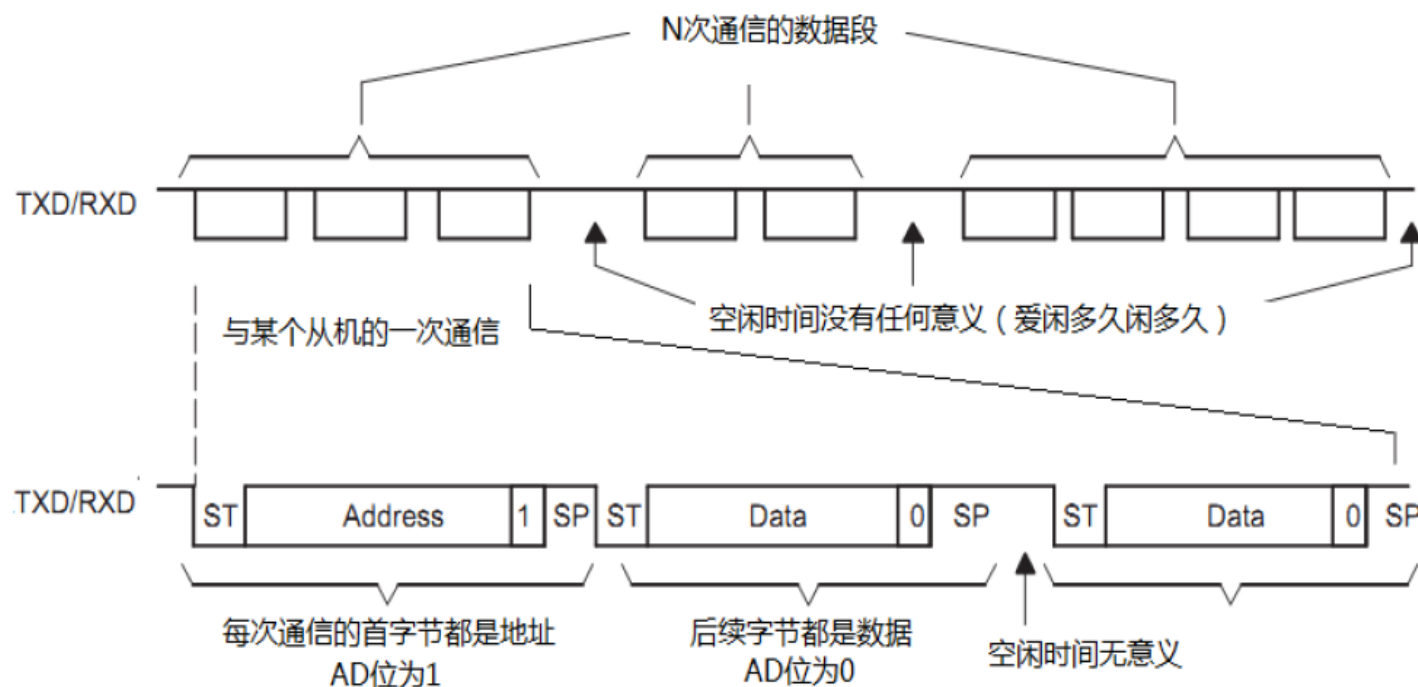
- 1) 当地址位是1时，说明本帧是地址帧，于是每个从机都“打起精神”，认真的比对地址是否与自己家的一样。
- 2) 如果与本机地址一样，则意味着接下来的数据帧是给自己的，可以打开“仓门”准备收货，当然也意味着自己也可以给主机“纳贡”。
- 3) 如果不是，则需等下一个地址帧的到来，这接下来的数据帧与自己无关，当然也不许发数据给主机。



9.2.1 异步串行通信基本概念



地址位模式 缺点： 如果主机与从机一次要通信很长时间，在这段时间内并不更换从机，但是这期间每一字节后都有一个地址位（一直为0），就造成了浪费。

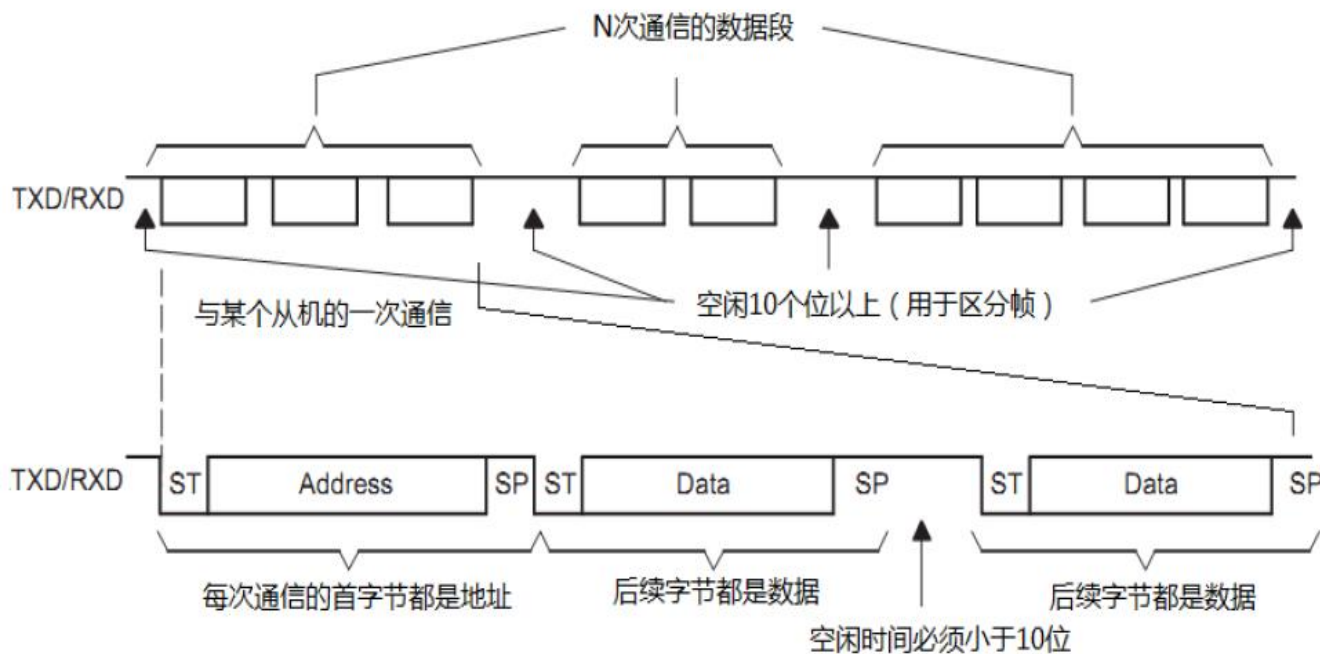


9.2.1 异步串行通信基本概念



空闲帧模式多机通信：

- 1) 默认第一帧一定是地址帧。
- 2) 接下来的帧都是数据帧，被选中地址的从机开始与主机双向通信。
- 3) 用连续10位以上的1表示本次通信结束。
- 4) 当各待命从机检测到10位以上的1的时候，就代表“新的机遇”来了，下一个帧是地址帧，说不定可以轮到自己“上场”呢。





第9章 异步串行通信

- 9.2.1 异步串行通信基本概念
- 9.2.2 MSP430的USCI

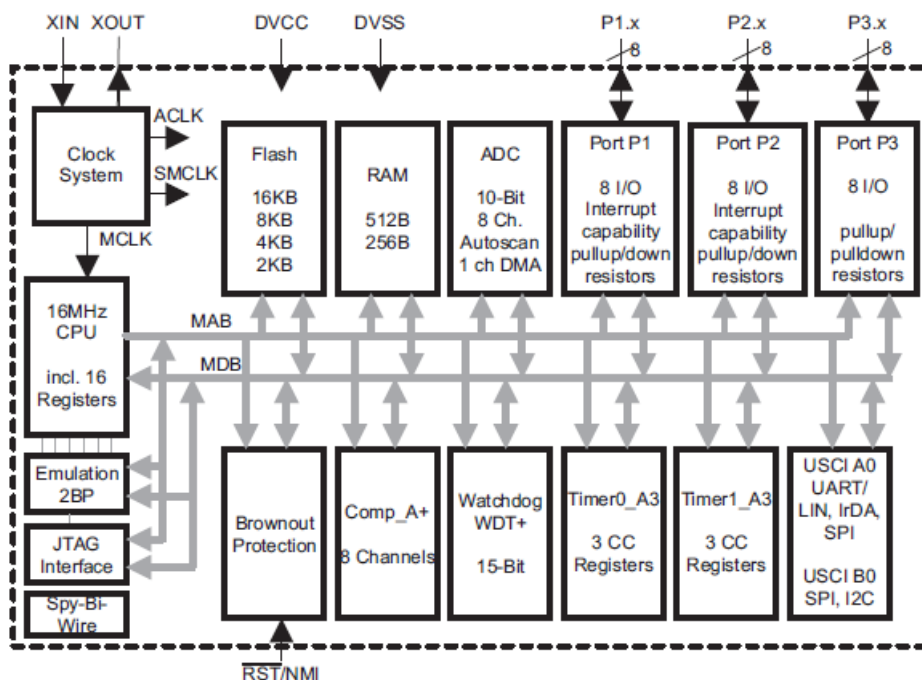
9.2.2 MSP430的USCI



USCI全称为universal serial communication interface，通用串行通信接口。
MSP430G2553单片机中带1个USCI_A模块和一个USCI_B模块。

USCI_A可配置为UART、LIN（局域互联网）、IrDA（红外通信）、SPI模式。

USCI_B可配置为SPI和I2C模式。



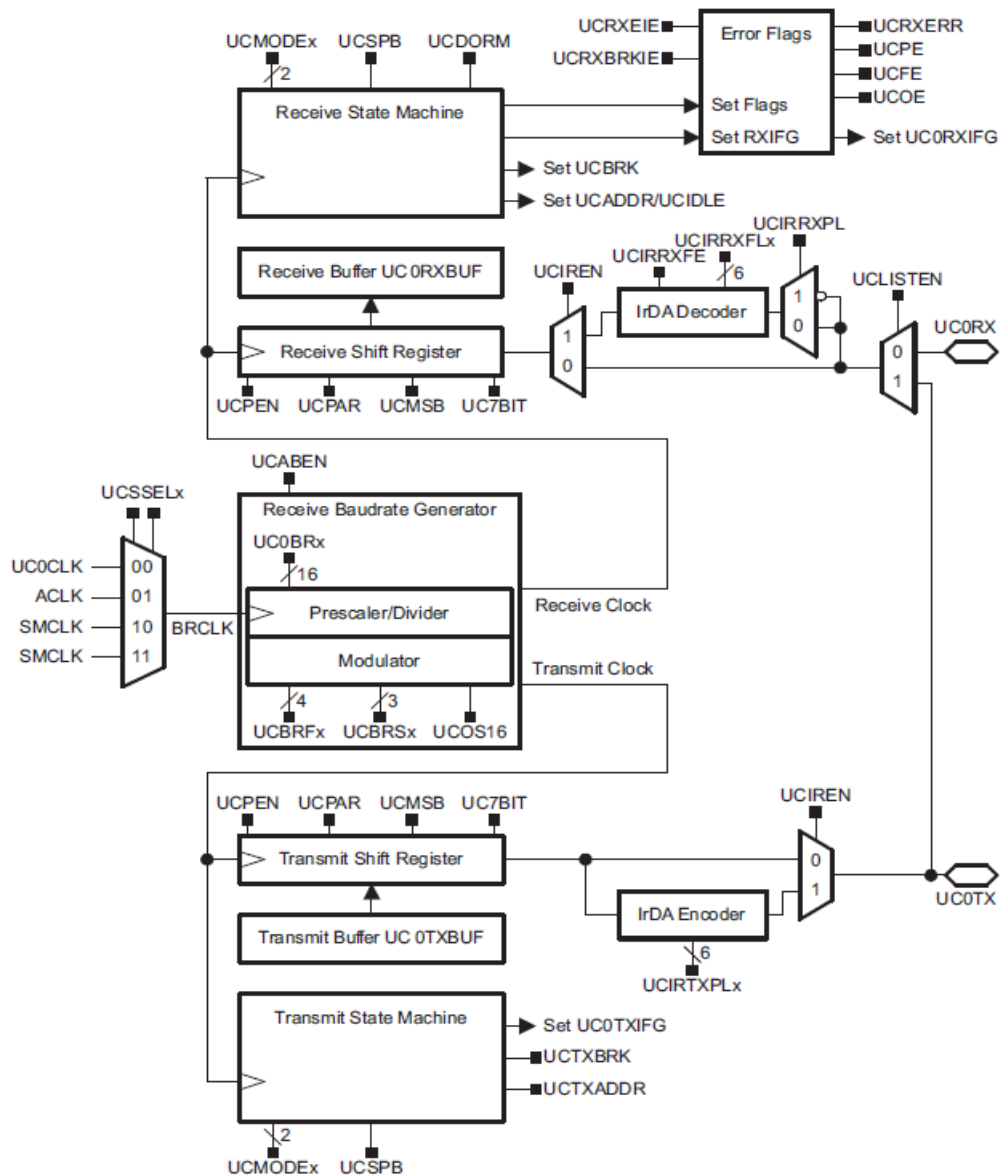
9.2.2 MSP430的USCI



有了USCI模块，UART串口通信将变得非常简单。

- 1) 硬件通信模块相当于邮局（USCI），你只要填好收寄地址人员电话等信息（USCI初始化），剩下的收发邮件（数据）都可以交给邮局（USCI）。
- 2) 发邮件只需将邮件（数据）放入TXBUF发送缓冲器，邮局自动的会发出邮件，发送完毕还会电话通知（中断）。
- 3) 收邮件只需查看自家邮箱接收缓冲器RXBUF就行，有邮件时，邮局同样会电话通知（中断）。
- 4) 所以，初始化完成后，只需操作发送缓冲区TXBUF和接收缓冲器RXBUF即可。

9.2.2 MSP430的USCI



串行异步通信特点

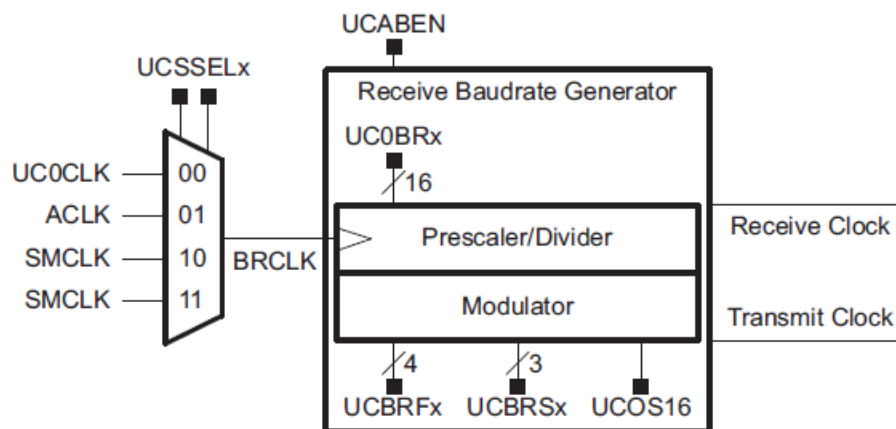
- 异步模式，包括线路空闲/地址位通信协议
- 两个独立移位寄存器：输入移位寄存器和输出移位寄存器
- 传输7位或8位数据，可采用奇校验或偶校验或者无校验
- 从最低位开始的数据发送和接收
- 可编程实现分频因子为整数或小数的波特率
- 独立的发送和接收中断
- 通过有效的起始位检测将MSP430从低功耗唤醒

Figure 15-1. USCI_Ax Block Diagram: UART Mode (UCSYNC = 0)

9.2.2 MSP430的USCI



波特率发生器



时钟源

UCA控制寄存器 UCAxCTL1(USCI_Ax control register)

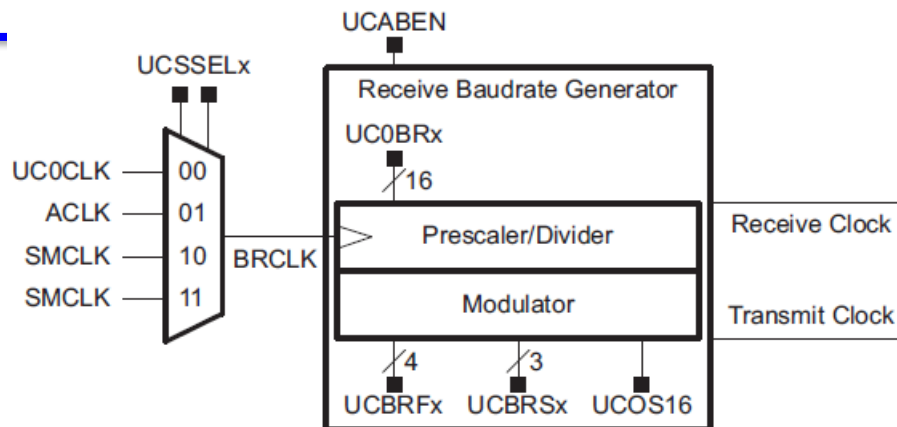


00: UCxCLK (外部时钟)
01: ACLK
10: SMCLK
11: SMCLK

9.2.2 MSP430的USCI



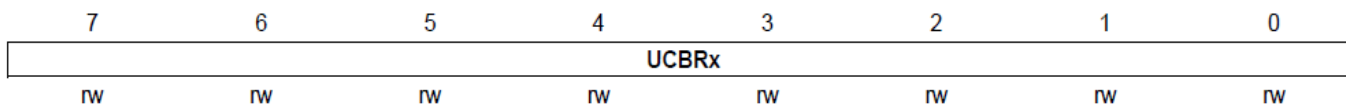
波特率设定



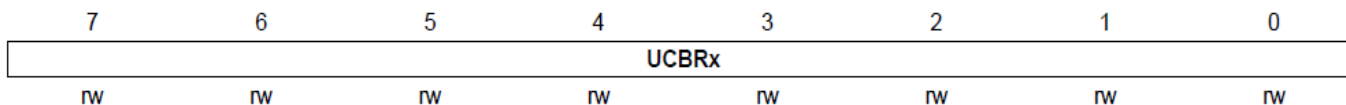
时钟源BRCLK需通过预分频器（整数倍分频）和调整器（小数分频）才能得到不同的波特率。

1) 设置预分频器： 预分频系数UCBRx由2个8位寄存器UCAxBR0和UCAxBR1构成

15.4.3 UCAxBR0, USCI_Ax Baud Rate Control Register 0



15.4.4 UCAxBR1, USCI_Ax Baud Rate Control Register 1



UCBRx

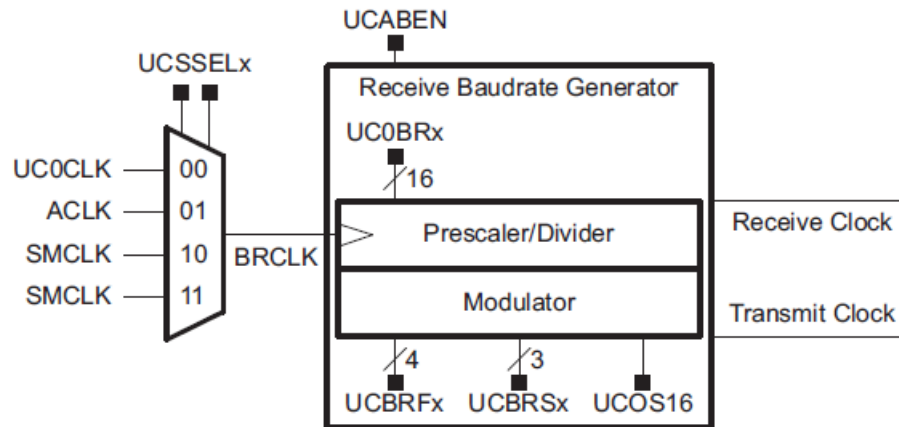
7-0

Clock prescaler setting of the Baud rate generator. The 16-bit value of (UCAxBR0 + UCAxBR1 × 256) forms the prescaler value.

9.2.2 MSP430的USCI



波特率设定



2) 设置调整器：调整器系数由调整寄存器UCAxMCTL确定

15.4.5 UCAxMCTL, USCI_Ax Modulation Control Register

7	6	5	4	3	2	1	0
UCBRFx				UCBRSx			UCOS16
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0
UCBRFx	Bits 7-4	First modulation stage select. These bits determine the modulation pattern for BITCLK16 when UCOS16 = 1. Ignored with UCOS16 = 0. Table 15-3 shows the modulation pattern.					
UCBRSx	Bits 3-1	Second modulation stage select. These bits determine the modulation pattern for BITCLK. Table 15-2 shows the modulation pattern.					
UCOS16	Bit 0	Oversampling mode enabled					
		0	Disabled				
		1	Enabled				



9.2.2 MSP430的USCI

2) 设置调整器：调整器系数由调整寄存器UCAxMCTL确定

15.4.5 UCAxMCTL, USCI_Ax Modulation Control Register

7	6	5	4	3	2	1	0
UCBRFx				UCBRSx			UCOS16
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0
UCBRFx	Bits 7-4	First modulation stage select. These bits determine the modulation pattern for BITCLK16 when UCOS16 = 1. Ignored with UCOS16 = 0. Table 15-3 shows the modulation pattern.					
UCBRSx	Bits 3-1	Second modulation stage select. These bits determine the modulation pattern for BITCLK. Table 15-2 shows the modulation pattern.					
UCOS16	Bit 0	Oversampling mode enabled					
		0	Disabled				
		1	Enabled				

UCOS16决定了波特率发生器的工作模式：低频模式（默认方式）；过采样模式。

在过采样模式下分两次调整，第一次调整系数由UCBRFx决定，第二次调整系数由UCBRSx决定。

9.2.2 MSP430的USCI



对于给定的BRCLK时钟源，所使用的波特率将决定分频因子 N ：

$$N = (f_{BRCLK} / f_{Baud})$$

式中， f_{BRCLK} 为实际进入波特率发生器的时钟频率

f_{Baud} 为实际所需要的波特率

$UCOS16 = 0$ 低频模式

$UCBR = INT(N)$ 预分频系数 取整

$UCBRS = round[(N - INT(N)) \times 8]$ 小数分频系数，四舍五入后取整

$UCOS16 = 1$ 过采样模式

$UCBR = INT(N/16)$ 预分频系数 取整

$UCBRF = round \{ [(N/16) - INT(N/16)] \times 16 \}$ 首次调整分频系数，

四舍五入后取整。一般经首次调整后波特率可基本达到要求，若要更高的精度需二次调整（参阅技术资料）。

[illegible]

[illegible]

9.2.2 MSP430的USCI



Table 15-4. Commonly Used Baud Rates, Settings, and Errors, UCOS16 = 0

BRCLK Frequency [Hz]	Baud Rate [Baud]	UCBRx	UCBRsX	UCBRFx	Maximum TX Error [%]		Maximum RX Error [%]	
32,768	1200	27	2	0	-2.8	1.4	-5.9	2.0
32,768	2400	13	6	0	-4.8	6.0	-9.7	8.3
32,768	4800	6	7	0	-12.1	5.7	-13.4	19.0
32,768	9600	3	3	0	-21.1	15.2	-44.3	21.3
1,048,576	9600	109	2	0	-0.2	0.7	-1.0	0.8
1,048,576	19200	54	5	0	-1.1	1.0	-1.5	2.5
1,048,576	38400	27	2	0	-2.8	1.4	-5.9	2.0
1,048,576	56000	18	6	0	-3.9	1.1	-4.6	5.7
1,048,576	115200	9	1	0	-1.1	10.7	-11.5	11.3
1,048,576	128000	8	1	0	-8.9	7.5	-13.8	14.8
1,048,576	256000	4	1	0	-2.3	25.4	-13.4	38.8
1,000,000	9600	104	1	0	-0.5	0.6	-0.9	1.2
1,000,000	19200	52	0	0	-1.8	0	-2.6	0.9
1,000,000	38400	26	0	0	-1.8	0	-3.6	1.8
1,000,000	56000	17	7	0	-4.8	0.8	-8.0	3.2
1,000,000	115200	8	6	0	-7.8	6.4	-9.7	16.1
1,000,000	128000	7	7	0	-10.4	6.4	-18.0	11.6
1,000,000	256000	3	7	0	-29.6	0	-43.6	5.2
4,000,000	9600	416	6	0	-0.2	0.2	-0.2	0.4
4,000,000	19200	208	3	0	-0.2	0.5	-0.3	0.8
4,000,000	38400	104	1	0	-0.5	0.6	-0.9	1.2
4,000,000	56000	71	4	0	-0.6	1.0	-1.7	1.3
4,000,000	115200	34	6	0	-2.1	0.6	-2.5	3.1
4,000,000	128000	31	2	0	-0.8	1.6	-3.6	2.0
4,000,000	256000	15	5	0	-4.0	3.2	-8.4	5.2
8,000,000	9600	833	2	0	-0.1	0	-0.2	0.1
8,000,000	19200	416	6	0	-0.2	0.2	-0.2	0.4
8,000,000	38400	208	3	0	-0.2	0.5	-0.3	0.8
8,000,000	56000	142	7	0	-0.6	0.1	-0.7	0.8
8,000,000	115200	69	4	0	-0.6	0.8	-1.8	1.1
8,000,000	128000	62	4	0	-0.8	0	-1.2	1.2
8,000,000	256000	31	2	0	-0.8	1.6	-3.6	2.0
12,000,000	9600	1250	0	0	0	0	-0.05	0.05
12,000,000	19200	625	0	0	0	0	-0.2	0
12,000,000	38400	312	4	0	-0.2	0	-0.2	0.2
12,000,000	56000	214	2	0	-0.3	0.2	-0.4	0.5

Table 15-4. Commonly Used Baud Rates, Settings, and Errors, UCOS16 = 0 (continued)

BRCLK Frequency [Hz]	Baud Rate [Baud]	UCBRx	UCBRsX	UCBRFx	Maximum TX Error [%]		Maximum RX Error [%]	
12,000,000	115200	104	1	0	-0.5	0.6	-0.9	1.2
12,000,000	128000	93	6	0	-0.8	0	-1.5	0.4
12,000,000	256000	46	7	0	-1.9	0	-2.0	2.0
16,000,000	9600	1666	6	0	-0.05	0.05	-0.05	0.1
16,000,000	19200	833	2	0	-0.1	0.05	-0.2	0.1
16,000,000	38400	416	6	0	-0.2	0.2	-0.2	0.4
16,000,000	56000	285	6	0	-0.3	0.1	-0.5	0.2
16,000,000	115200	138	7	0	-0.7	0	-0.8	0.6
16,000,000	128000	125	0	0	0	0	-0.8	0
16,000,000	256000	62	4	0	-0.8	0	-1.2	1.2

9.2.2 MSP430的USCI



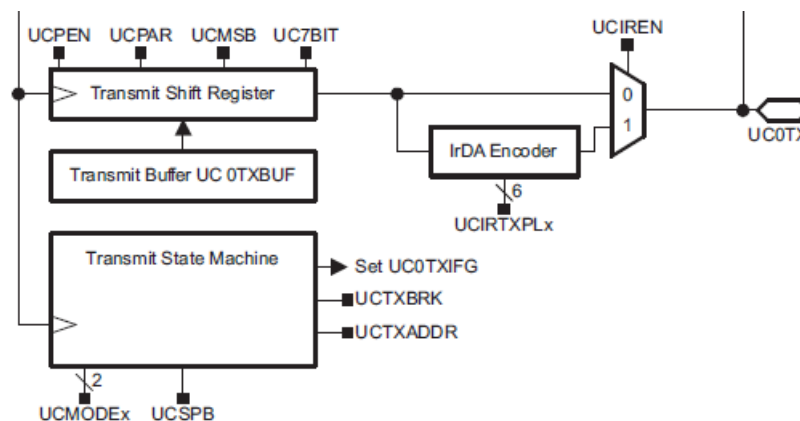
Table 15-5. Commonly Used Baud Rates, Settings, and Errors, UCOS16 = 1

BRCLK Frequency [Hz]	Baud Rate [Baud]	UCBRx	UCBR5x	UCBRFx	Maximum TX Error [%]		Maximum RX Error [%]	
1,048,576	9600	6	0	13	-2.3	0	-2.2	0.8
1,048,576	19200	3	1	6	-4.6	3.2	-5.0	4.7
1,000,000	9600	6	0	8	-1.8	0	-2.2	0.4
1,000,000	19200	3	0	4	-1.8	0	-2.6	0.9
1,000,000	57600	1	7	0	-34.4	0	-33.4	0
4,000,000	9600	26	0	1	0	0.9	0	1.1
4,000,000	19200	13	0	0	-1.8	0	-1.9	0.2
4,000,000	38400	6	0	8	-1.8	0	-2.2	0.4
4,000,000	57600	4	5	3	-3.5	3.2	-1.8	6.4
4,000,000	115200	2	3	2	-2.1	4.8	-2.5	7.3
4,000,000	230400	1	7	0	-34.4	0	-33.4	0
8,000,000	9600	52	0	1	-0.4	0	-0.4	0.1
8,000,000	19200	26	0	1	0	0.9	0	1.1
8,000,000	38400	13	0	0	-1.8	0	-1.9	0.2
8,000,000	57600	8	0	11	0	0.88	0	1.6
8,000,000	115200	4	5	3	-3.5	3.2	-1.8	6.4
8,000,000	230400	2	3	2	-2.1	4.8	-2.5	7.3
8,000,000	460800	1	7	0	-34.4	0	-33.4	0
12,000,000	9600	78	0	2	0	0	-0.05	0.05
12,000,000	19200	39	0	1	0	0	0	0.2
12,000,000	38400	19	0	8	-1.8	0	-1.8	0.1
12,000,000	57600	13	0	0	-1.8	0	-1.9	0.2
12,000,000	115200	6	0	8	-1.8	0	-2.2	0.4
12,000,000	230400	3	0	4	-1.8	0	-2.6	0.9
16,000,000	9600	104	0	3	0	0.2	0	0.3
16,000,000	19200	52	0	1	-0.4	0	-0.4	0.1
16,000,000	38400	26	0	1	0	0.9	0	1.1
16,000,000	57600	17	0	6	0	0.9	-0.1	1.0
16,000,000	115200	8	0	11	0	0.9	0	1.6
16,000,000	230400	4	5	3	-3.5	3.2	-1.8	6.4
16,000,000	460800	2	3	2	-2.1	4.8	-2.5	7.3

9.2.2 MSP430的USCI



发送部件



相关控制位集中在控制寄存器
UCAxCTL0和
UCAxCTL1

7	6	5	4	3	2	1	0
UCPEN	UCPAR	UCMSB	UC7BIT	UCSPB	UCMODEx		UCSYNC
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0
UCPEN	Bit 7	Parity enable					
		0 Parity disabled.					
		1 Parity enabled. Parity bit is generated (UCAxTXD) and expected (UCAxRXD). In address-bit multiprocessor mode, the address bit is included in the parity calculation.					
UCPAR	Bit 6	Parity select. UCPAR is not used when parity is disabled.					
		0 Odd parity					
		1 Even parity					
UCMSB	Bit 5	MSB first select. Controls the direction of the receive and transmit shift register.					
		0 LSB first					
		1 MSB first					
UC7BIT	Bit 4	Character length. Selects 7-bit or 8-bit character length.					
		0 8-bit data					
		1 7-bit data					
UCSPB	Bit 3	Stop bit select. Number of stop bits.					
		0 One stop bit					
		1 Two stop bits					
UCMODEx	Bits 2-1	USCI mode. The UCMODEx bits select the asynchronous mode when UCSYNC = 0.					
		00 UART mode					
		01 Idle-line multiprocessor mode					
		10 Address-bit multiprocessor mode					
		11 UART mode with automatic baud rate detection					
UCSYNC	Bit 0	Synchronous mode enable					
		0 Asynchronous mode					
		1 Synchronous mode					

9.2.2 MSP430的USCI



15.4.2 UCxCTL1, USCI_Ax Control Register 1

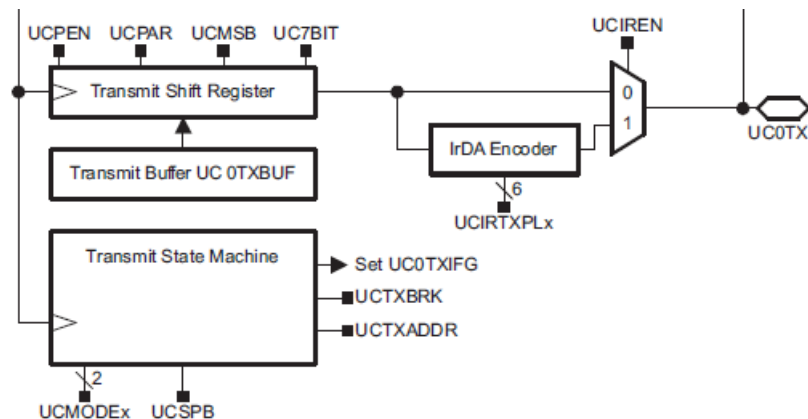
7	6	5	4	3	2	1	0
UCSSELx		UCRXEIE	UCBRKIE	UCDORM	UCTXADDR	UCTXBRK	UCSWRST
rw-0		rw-0	rw-0	rw-0	rw-0	rw-0	rw-1

UCSSELx	Bits 7-6	USCI clock source select. These bits select the BRCLK source clock.	
		00	UCLK
		01	ACLK
		10	SMCLK
		11	SMCLK
UCRXEIE	Bit 5	Receive erroneous-character interrupt-enable	
		0	Erroneous characters rejected and UCAxRXIFG is not set
		1	Erroneous characters received will set UCAxRXIFG
UCBRKIE	Bit 4	Receive break character interrupt-enable	
		0	Received break characters do not set UCAxRXIFG.
		1	Received break characters set UCAxRXIFG.
UCDORM	Bit 3	Dormant. Puts USCI into sleep mode.	
		0	Not dormant. All received characters will set UCAxRXIFG.
		1	Dormant. Only characters that are preceded by an idle-line or with address bit set will set UCAxRXIFG. In UART mode with automatic baud rate detection only the combination of a break and synch field will set UCAxRXIFG.
UCTXADDR	Bit 2	Transmit address. Next frame to be transmitted will be marked as address depending on the selected multiprocessor mode.	
		0	Next frame transmitted is data
		1	Next frame transmitted is an address
UCTXBRK	Bit 1	Transmit break. Transmits a break with the next write to the transmit buffer. In UART mode with automatic baud rate detection 055h must be written into UCAxTXBUF to generate the required break/synch fields. Otherwise 0h must be written into the transmit buffer.	
		0	Next frame transmitted is not a break
		1	Next frame transmitted is a break or a break/synch
UCSWRST	Bit 0	Software reset enable	
		0	Disabled. USCI reset released for operation.
		1	Enabled. USCI logic held in reset state.

9.2.2 MSP430的USCI



发送流程

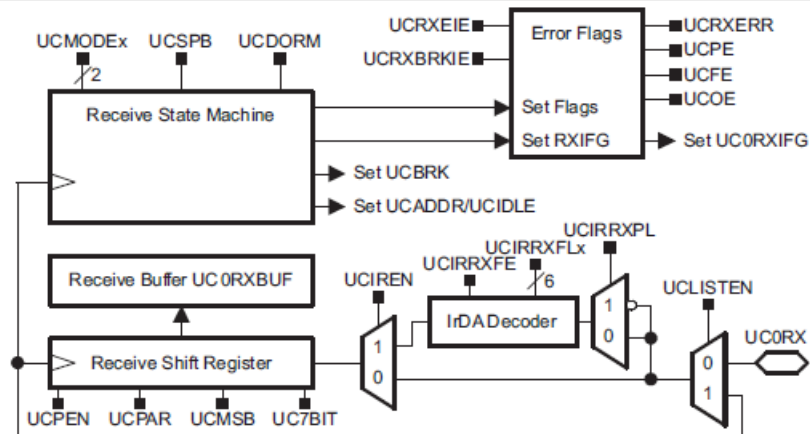


- 1) 清除UCSWRST使USCI模块进入工作状态;
- 2) 当有数据写入UCAxTXBUF中时, 比特率发生器开始输出发送时钟;
- 3) UCAxTXBUF中的数据随即被移入发送移位寄存器, 然后在发送时钟的节拍下将配置好的数据帧依次串行地发送出去;
- 4) 每次发送完毕后相应的中断标志位置1;
- 5) 如果一次发送结束时UCAxTXBUF又被写入新的有效数据, 则随即开始下一次的发送;
- 6) 否则, 发送部件返回到空闲态, 并关闭波特率发生器。

9.2.2 MSP430的USCI



接收部件



控制位与发送部件相同。
不同之处主要集中在状态
寄存器UCAxSTAT和控制
寄存器UCAxCTL1

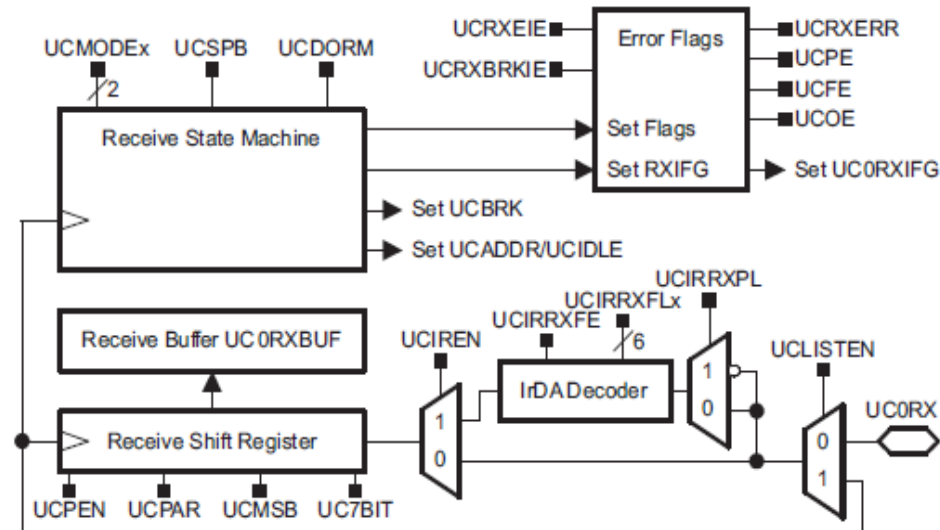
7	6	5	4	3	2	1	0
UCLISTEN	UCFE	UCOE	UCPE	UCBRK	UCRXERR	UCADDR UCIDLE	UCBUSY
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	r-0

UCLISTEN	Bit 7	Listen enable. The UCLISTEN bit selects loopback mode. 0 Disabled 1 Enabled. UCAxTXD is internally fed back to the receiver.
UCFE	Bit 6	Framing error flag 0 No error 1 Character received with low stop bit
UCOE	Bit 5	Overrun error flag. This bit is set when a character is transferred into UCAxRXBUF before the previous character was read. UCOE is cleared automatically when UCxRXBUF is read, and must not be cleared by software. Otherwise, it will not function correctly. 0 No error 1 Overrun error occurred
UCPE	Bit 4	Parity error flag. When UCPEN = 0, UCPE is read as 0. 0 No error 1 Character received with parity error
UCBRK	Bit 3	Break detect flag 0 No break condition 1 Break condition occurred
UCRXERR	Bit 2	Receive error flag. This bit indicates a character was received with error(s). When UCRXERR = 1, on or more error flags (UCFE, UCPE, UCOE) is also set. UCRXERR is cleared when UCAxRXBUF is read. 0 No receive errors detected 1 Receive error detected
UCADDR	Bit 1	Address received in address-bit multiprocessor mode. 0 Received character is data 1 Received character is an address
UCIDLE		Idle line detected in idle-line multiprocessor mode. 0 No idle line detected 1 Idle line detected
UCBUSY	Bit 0	USCI busy. This bit indicates if a transmit or receive operation is in progress. 0 USCI inactive 1 USCI transmitting or receiving

9.2.2 MSP430的USCI



接收流程



- 1) 清除UCSWRST使USCI模块进入工作状态，同时接收部件准备好并处于空闲状态；
- 2) 当检测到下降沿时波特率发生器开始工作，同时检测是否是一个有效的数据帧起始位。如果未检测到则返回空闲态，同时关闭波特率发生器；
- 3) 如果检测到是一个有效的数据帧起始位，则开始接收；
- 4) 接收完毕后中断标志位UCA0RXIFG置1。

9.2.2 MSP430的USCI



USCI的中断

15.4.12 IE2, Interrupt Enable Register 2

7	6	5	4	3	2	1	0
						UCA0TXIE	UCA0RXIE
						rw-0	rw-0

Bits 7-2 These bits may be used by other modules (see the device-specific data sheet).

UCA0TXIE Bit 1 USCI_A0 transmit interrupt enable

0 Interrupt disabled

1 Interrupt enabled

UCA0RXIE Bit 0 USCI_A0 receive interrupt enable

0 Interrupt disabled

1 Interrupt enabled

15.4.13 IFG2, Interrupt Flag Register 2

7	6	5	4	3	2	1	0
						UCA0TXIFG	UCA0RXIFG
						rw-1	rw-0

Bits 7-2 These bits may be used by other modules (see the device-specific data sheet).

UCA0TXIFG Bit 1 USCI_A0 transmit interrupt flag. UCA0TXIFG is set when UCA0TXBUF is empty.

0 No interrupt pending

1 Interrupt pending

UCA0RXIFG Bit 0 USCI_A0 receive interrupt flag. UCA0RXIFG is set when UCA0RXBUF has received a complete character.

0 No interrupt pending

1 Interrupt pending



9.2.2 MSP430的USCI

串口中断向量: USCIAB0RX_VECTOR
USCIAB0TX_VECTOR

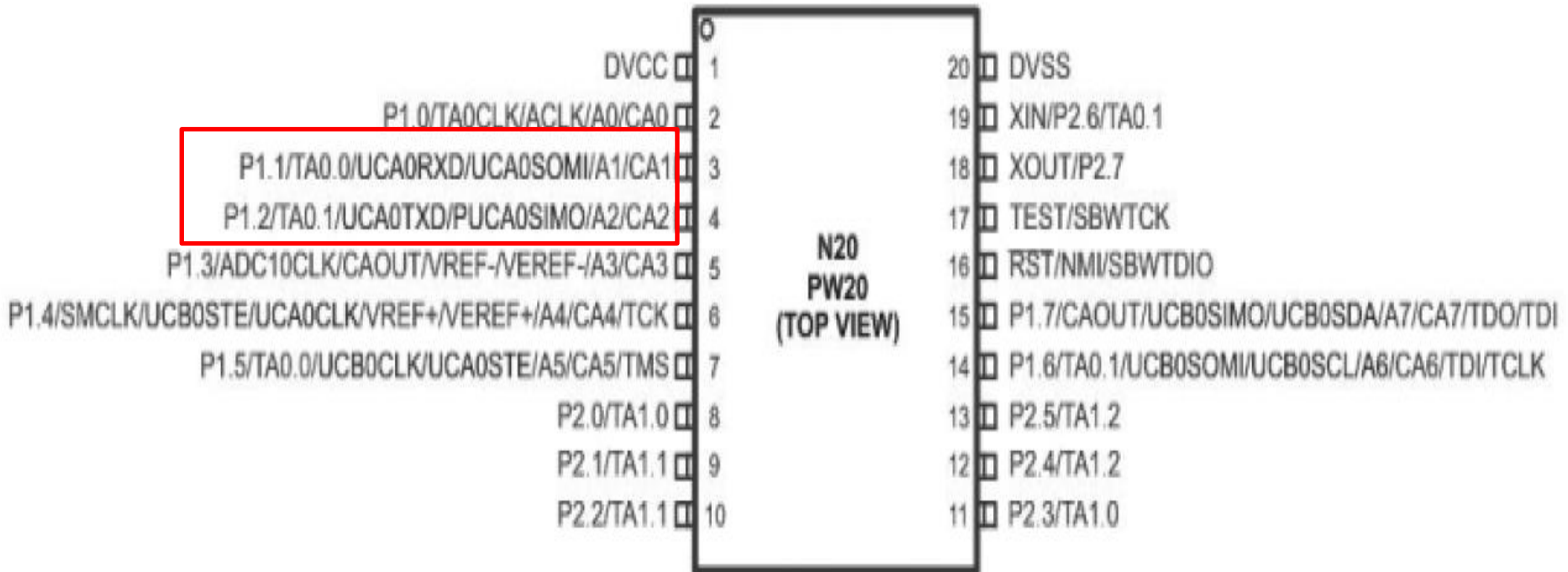
MSP430G2553单片机中带1个USCI_A模块和一个USCI_B模块。尽管两部分功能不尽相同，但共用一个发送中断向量和接收中断向量。

- **USCI模块初始化**

1. 设置UCSWRST=1
2. 初始化所有的USCI寄存器
3. 配置端口
4. 软件清除UCSWRST
5. 设置UCRXIE或UCTXIE 使能中断



9.2.2 MSP430的USCI



9.2.2 MSP430的USCI



例1:

```
#include <msp430g2553.h>
```

```
void main( )
```

```
{
```

```
    WDTCTL = WDTPW + WDTHOLD;
```

```
    UCA0CTL1 |= UCSWRST;
```

```
    P1SEL |= BIT1 + BIT2 ;
```

```
    P1SEL2 |= BIT1 + BIT2 ;
```

```
    UCA0CTL1 |= UCSSEL_2;
```

```
    UCA0BR0 = 104;
```

```
    UCA0BR1 = 0;
```

```
    UCA0MCTL = UCBRS_1;
```

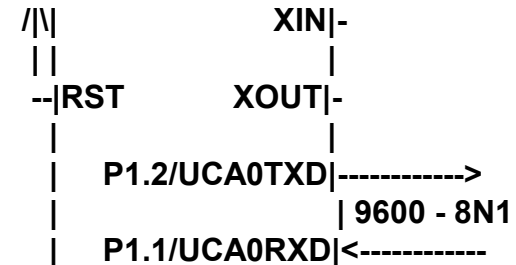
```
    UCA0CTL1 &= ~UCSWRST;
```

```
    IE2 |= UCA0RXIE;
```

```
    __bis_SR_register(LPM0_bits + GIE);    // Enter LPM0, interrupts enabled
```

```
}
```

MSP430G2xx3



```
// P1.1 = RXD, P1.2=TXD
```

```
// P1.1 = RXD, P1.2=TXD
```

```
// SMCLK
```

```
// 1MHz 9600
```

```
// 1MHz 9600
```

```
// Modulation UCBRSx = 1
```

```
// **Initialize USCI state machine**
```

```
// Enable USCI_A0 RX interrupt
```


9.2.2 MSP430的USCI



```
#pragma vector=USCIAB0RX_VECTOR
__interrupt void USCI0RX_ISR(void)
{
    while (!(IFG2&UCA0TXIFG));           // USCI_A0 TX buffer ready?
    UCA0TXBUF = UCA0RXBUF;                // TX -> RXed character
}
```

9.2.2 MSP430的USCI

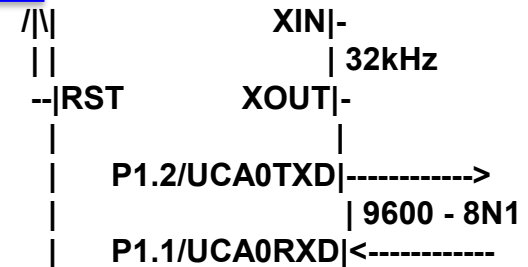
MSP430G2xx3



例2:

```
#include <msp430g2553.h>

const char string1[] = { "Hello World\r\n" };
unsigned int i;
int main(void)
{
    WDTCTL = WDTPW + WDTHOLD;           // Stop WDT
    UCA0CTL1 |= UCSWRST;
    P1SEL = BIT1 + BIT2 ;                // P1.1 = RXD, P1.2=TXD
    P1SEL2 = BIT1 + BIT2 ;               // P1.1 = RXD, P1.2=TXD
    UCA0CTL1 |= UCSSEL_1;                // CLK = ACLK
    UCA0BR0 = 0x03;                      // 32kHz/9600 = 3.41
    UCA0BR1 = 0x00;
    UCA0MCTL = UCBRS_3;                  // Modulation UCBRSx = 3
    UCA0CTL1 &= ~UCSWRST;                // **Initialize USCI state machine**
    IE2 |= UCA0RXIE;                    // Enable USCI_A0 RX interrupt
    __bis_SR_register(LPM3_bits + GIE);  // Enter LPM3 w/ int until Byte RXed
}
```



9.2.2 MSP430的USCI



```
#pragma vector=USCIAB0TX_VECTOR
__interrupt void USCI0TX_ISR(void)
{
    UCA0TXBUF = string1[i++];           // TX next character

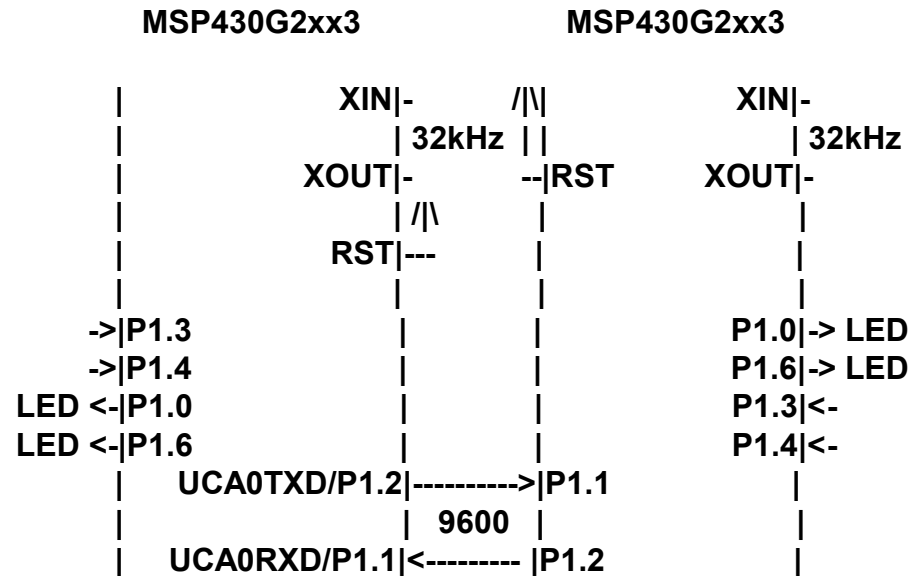
    if (i == sizeof string1 - 1)        // TX over?
        IE2 &= ~UCA0TXIE;              // Disable USCI_A0 TX interrupt
}

#pragma vector=USCIAB0RX_VECTOR
__interrupt void USCI0RX_ISR(void)
{
    if (UCA0RXBUF == 'u')                // 'u' received?
    {
        i = 0;
        IE2 |= UCA0TXIE;                // Enable USCI_A0 TX interrupt
        UCA0TXBUF = string1[i++];
    }
}
```

9.2.2 MSP430的USCI



例3:



9.2.2 MSP430的USCI



```
#include <msp430g2553.h>

void main( )
{
    WDTCTL = WDTPW + WDTHOLD;           // Stop watchdog timer
    P1DIR = BIT0 + BIT6;                 // P1.0/6 setup for LED output
    P1OUT &= ~ BIT0 + BIT6;
    UCA0CTL1 |= UCSWRST;
    P1SEL = BIT1 + BIT2 ;                // P1.1 = RXD, P1.2=TXD
    P1SEL2 = BIT1 + BIT2;
    UCA0CTL1 |= UCSSEL_1;                // CLK = ACLK
    UCA0BR0 = 3;                         // 32kHz/9600 = 3.41
    UCA0BR1 = 0;
    UCA0MCTL = UCBRS_3;                  // Modulation UCBRSx = 3
    UCA0CTL1 &= ~UCSWRST;                 // **Initialize USCI state machine**
    IE2 |= UCA0RXIE + UCA0TXIE;         // Enable USCI_A0 TX/RX interrupt

    __bis_SR_register(LPM3_bits + GIE);  // Enter LPM3 w/ interrupts enabled
}
```

9.2.2 MSP430的USCI



```
// USCI A0/B0 Transmit ISR
#pragma vector=USCIAB0TX_VECTOR
__interrupt void USCI0TX_ISR(void)
{
    unsigned char TxByte=0;
    if (P1IN & BIT3)
        TxByte |= BIT6;
    if (P1IN & BIT4)
        TxByte |= BIT0;
    UCA0TXBUF = TxByte;           // Read, justify, and transmit
}
```

```
// USCI A0/B0 Receive ISR
#pragma vector=USCIAB0RX_VECTOR
__interrupt void USCI0RX_ISR(void)
{
    P1OUT = UCA0RXBUF;           // Display RX'ed charater
}
```